# SQL BASIC ASSIGNMENT

## Question 1: Create a database named `company_db` and a table named `employees` with the required fields.

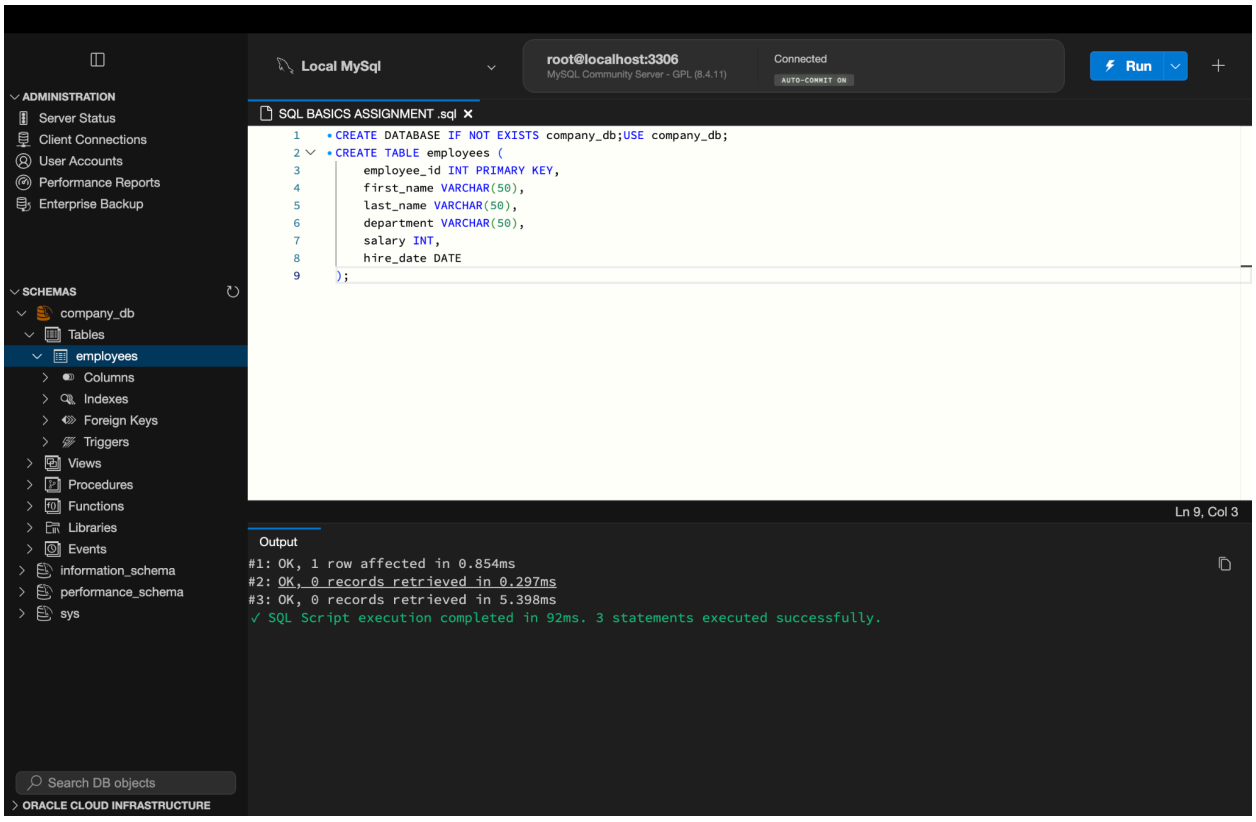
**SQL Query:**

```
CREATE DATABASE IF NOT EXISTS company_db;
USE company_db;

CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    department VARCHAR(50),
    salary INT,
    hire_date DATE
);
```

**Practical Output:**

The database `company_db` and the table `employees` were created successfully in MySQL Workbench.



# Question 2: Insert Five Employee Records into the Employees Table

**SQL Query:**

```
USE company_db;


INSERT INTO employees

(employee_id, first_name, last_name, department, salary, hire_date)

VALUES

(101, 'Amit', 'Sharma', 'HR', 50000, '2020-01-15'),

(102, 'Riya', 'Kapoor', 'Sales', 75000, '2019-03-22'),

(103, 'Raj', 'Mehta', 'IT', 90000, '2018-07-11'),

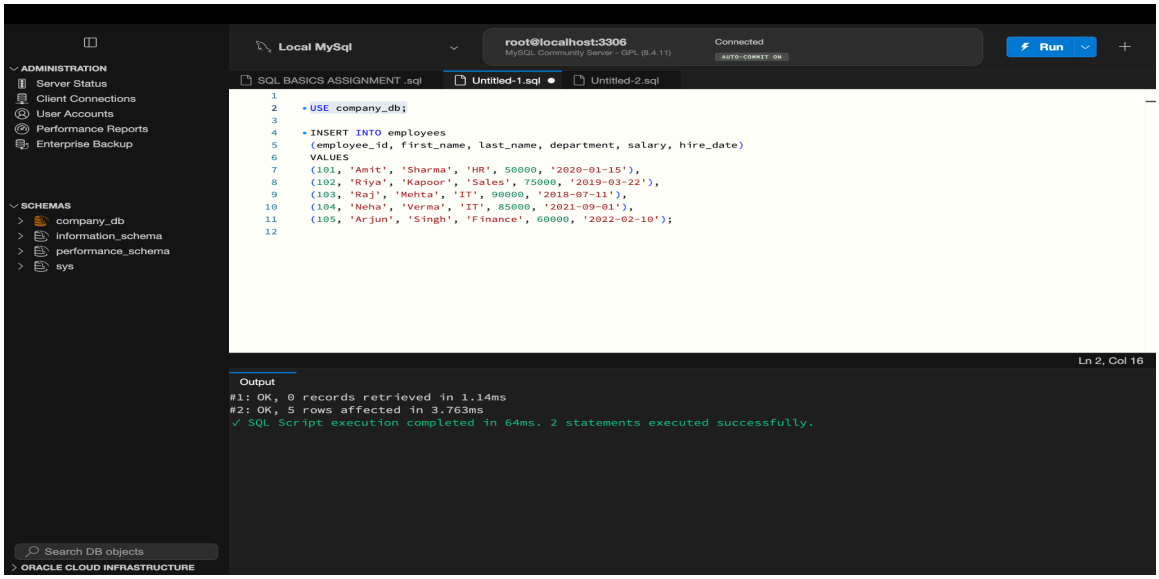
(104, 'Neha', 'Verma', 'IT', 85000, '2021-09-01'),

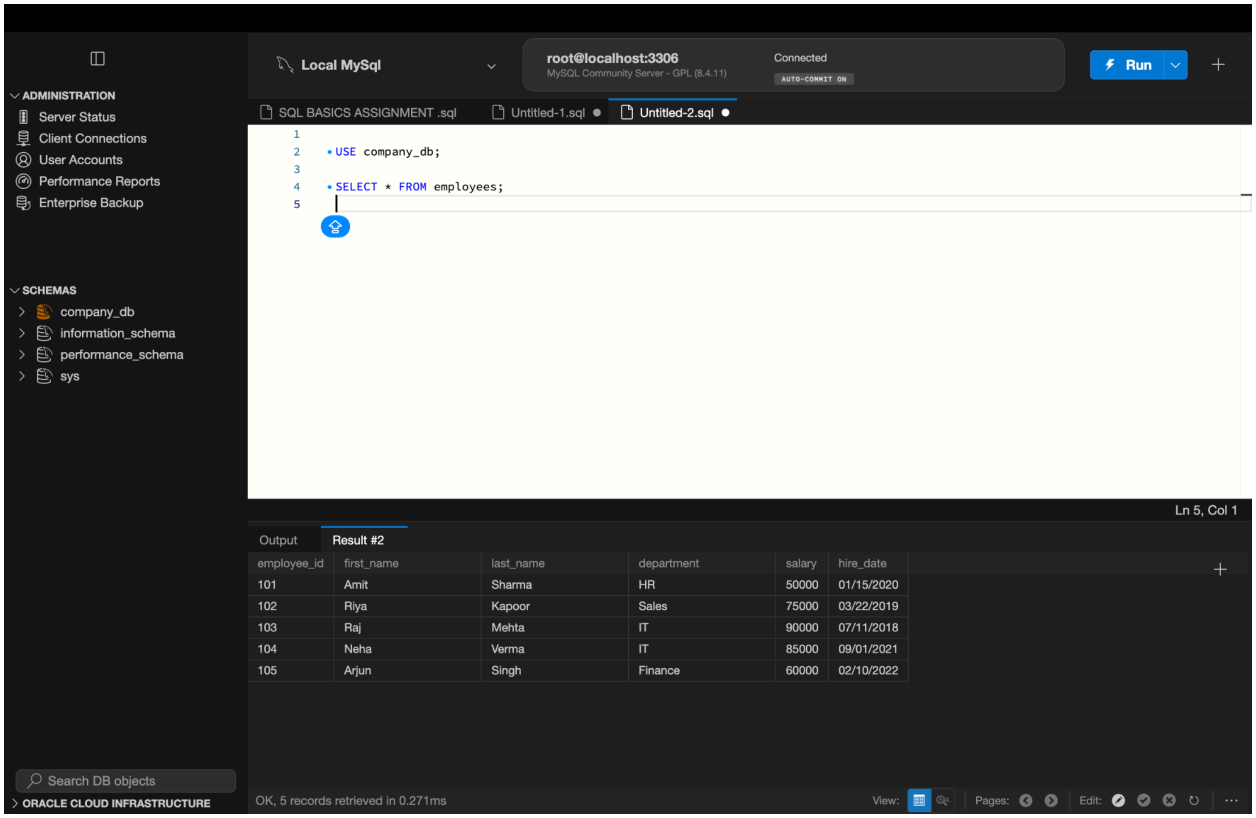
(105, 'Arjun', 'Singh', 'Finance', 60000, '2022-02-10');
```

**Output:**

The five employee records were successfully inserted into the `employees` table. MySQL confirmed that 5 rows were affected.

**Practical Proof:**






# Question 3: Display All Employees in Ascending Order of Salary

**SQL Query:**

USE company_db;

SELECT *

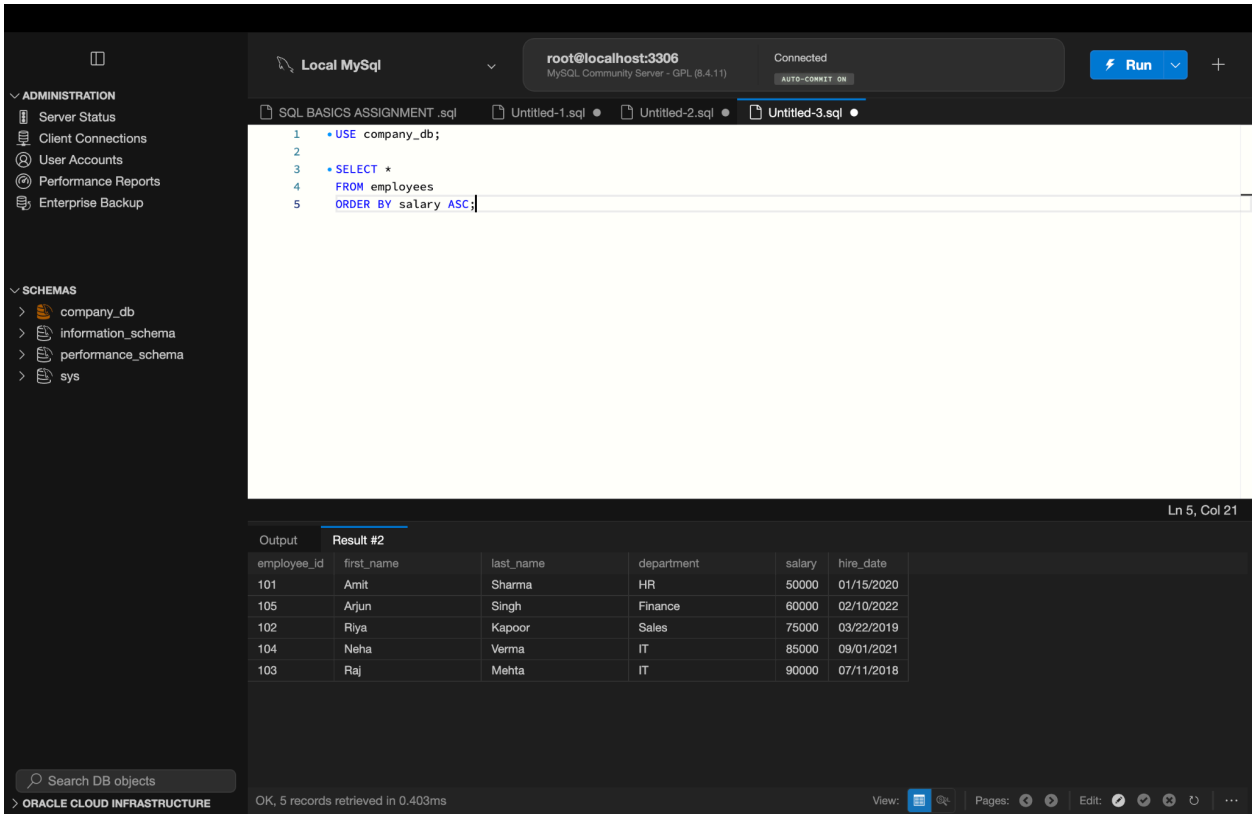
FROM employees

ORDER BY salary ASC;

**Output:**

The query displays all employee records sorted by salary in ascending order, from the lowest salary to the highest salary.

**PRACTICAL PROOF :**




# Question 4: Display Employees Sorted by Department in Ascending Order and Salary in Descending Order

**SQL Query:**

USE company_db;

SELECT *

FROM employees

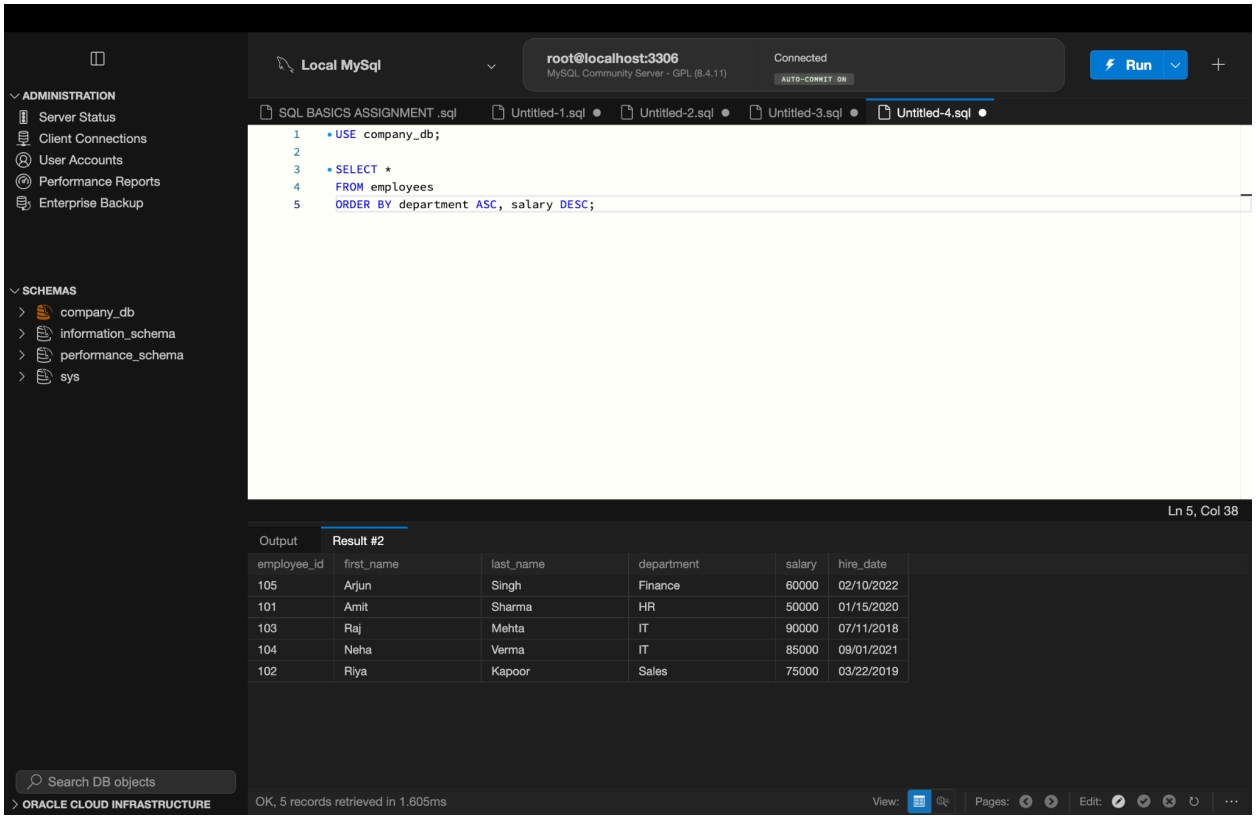
ORDER BY department ASC, salary DESC;

**Explanation:**

The `ORDER BY department ASC` clause sorts employees alphabetically by department. Within each department, `salary DESC` sorts employees from the highest salary to the lowest salary.

**Result:** Employees are displayed alphabetically by department, and employees in the same department are arranged by salary in descending order.

**PRACTICAL PROOF :**




# Question 5: Display Employees Working in the IT Department, Ordered by Hire Date in Descending Order

**SQL Query:**

USE company_db;

SELECT *

FROM employees

WHERE department = 'IT'

ORDER BY hire_date DESC;

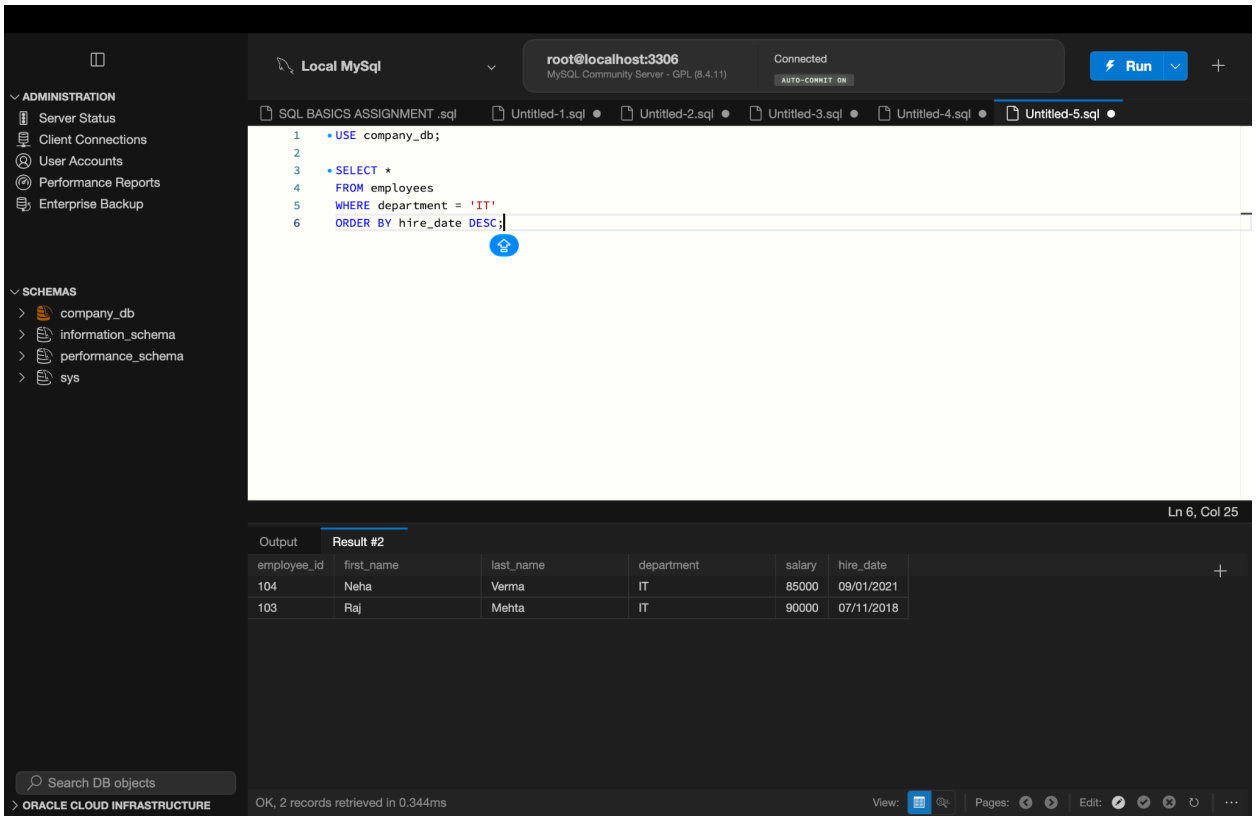
**Explanation:**

The `WHERE department = 'IT'` clause selects only employees who work in the IT department. The `ORDER BY hire_date DESC` clause sorts these employees by hire date, from the most recent to the earliest.

**Result:**

Only two employees belong to the IT department: Neha Verma and Raj Mehta. Neha appears first because she was hired more recently.

**Practical Proof:**




# Question 6: Create a Sales Table and Insert Five Records

**SQL Query:**

```
USE company_db;


CREATE TABLE sales (
    sale_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
```

```
    amount INT,
    sale_date DATE
);

INSERT INTO sales (sale_id, customer_name, amount, sale_date)
VALUES
(1, 'Aditi', 1500, '2024-08-01'),
(2, 'Rohan', 2200, '2024-08-03'),
(3, 'Aditi', 3500, '2024-09-05'),
(4, 'Meena', 2700, '2024-09-15'),
(5, 'Rohan', 4500, '2024-09-25');
```

**Explanation:**

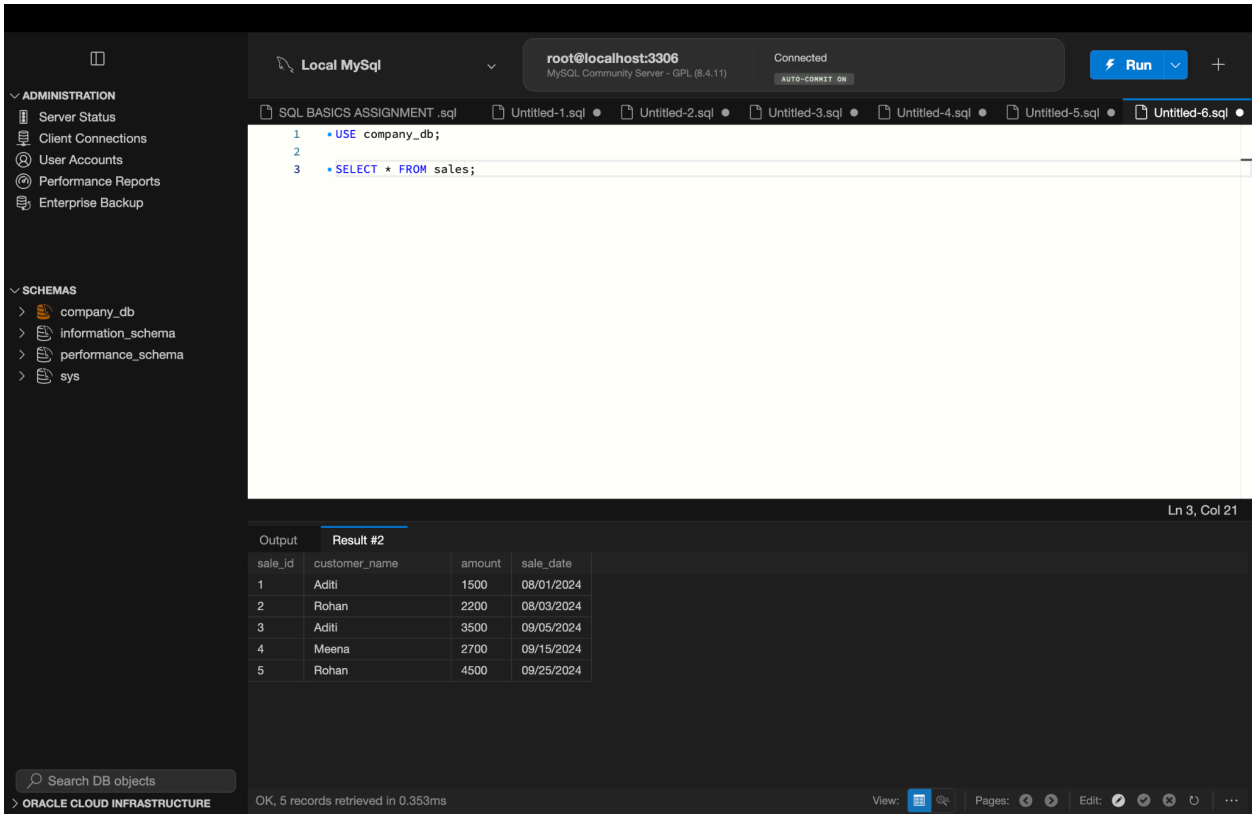
**The `CREATE TABLE` statement creates a table named `sales` with four columns: `sale_id`, `customer_name`, `amount`, and `sale_date`. The `sale_id` column is the primary key, which uniquely identifies each sale.**

**The `INSERT INTO` statement adds five sales records to the table.**

**Result:**

**The `sales` table is created, and five sales records are inserted successfully.**

**Practical Proof:**




# Question 7: Display Sales Records in Descending Order of Amount

**SQL Query:**

**USE company_db;**

**SELECT ***

**FROM sales**

**ORDER BY amount DESC;**

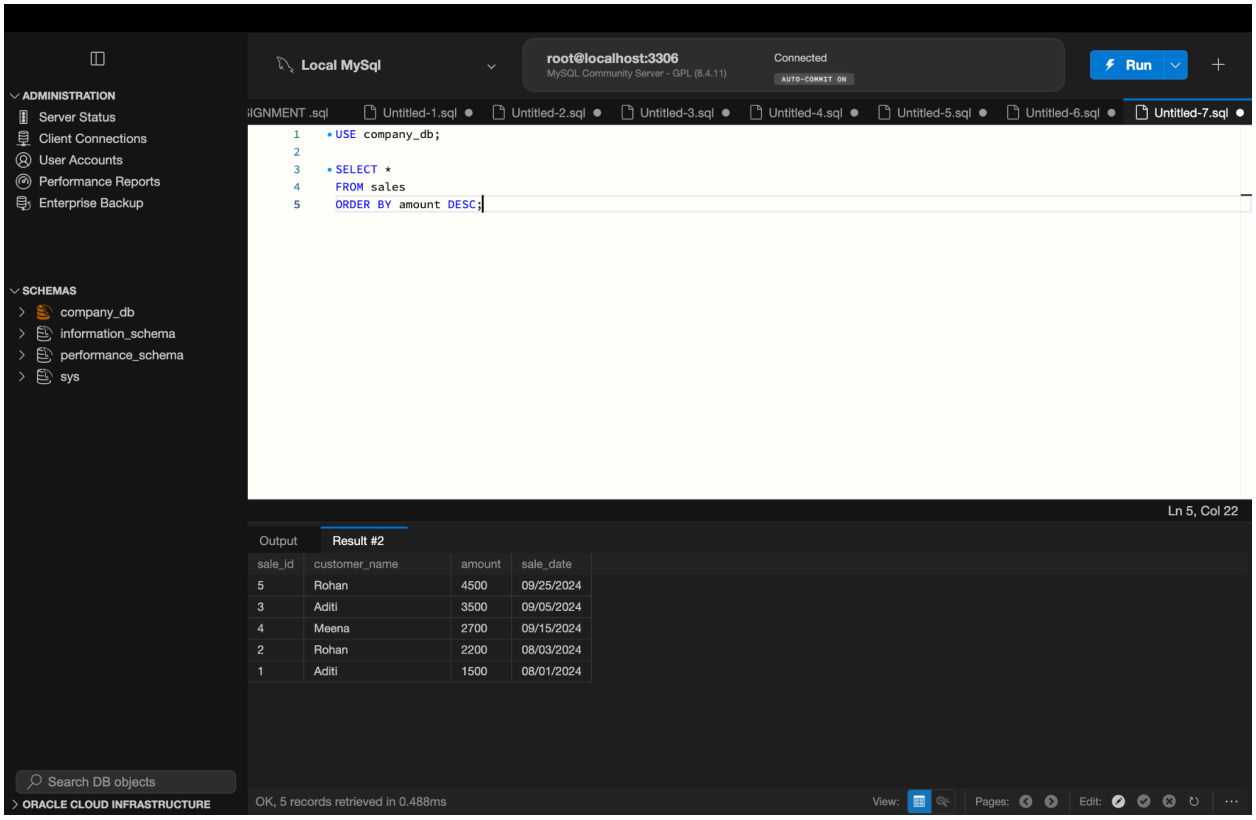
**Explanation:**

**The `SELECT *` statement displays all records from the `sales` table. The `ORDER BY amount DESC` clause sorts the records by sale amount in descending order, from the highest amount to the lowest.**

**Result:**

**The sales records are displayed from the highest sale amount (4500) to the lowest sale amount (1500).**

**Practical Proof:**




# Question 8: Display Sales Records for Customer Aditi

**SQL Query:**

**USE company_db;**

**SELECT ***

**FROM sales**

**WHERE customer_name = 'Aditi';**

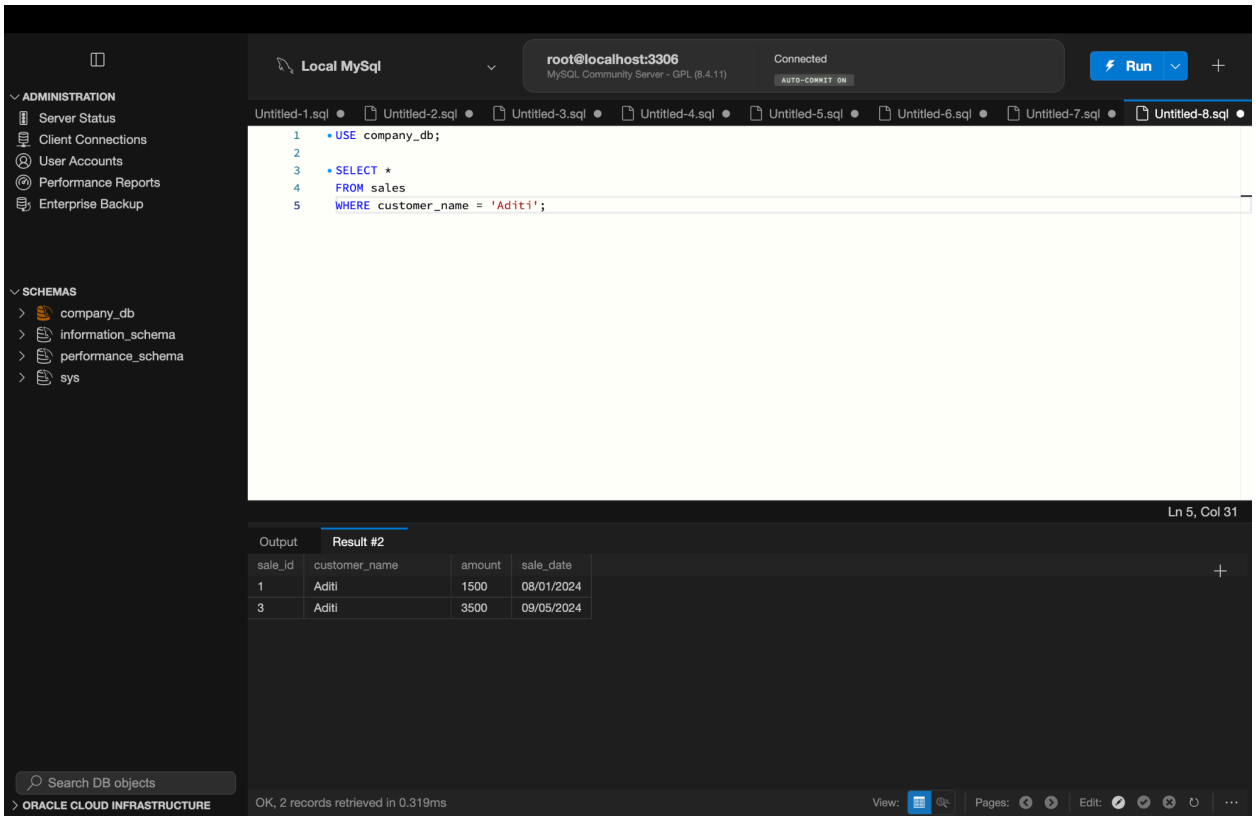
**Output:Explanation:**

**The `SELECT *` statement displays all columns from the `sales` table. The `WHERE customer_name = 'Aditi'` condition filters the records and displays only the sales made by Aditi.**

**Result:**

**Two sales records belonging to Aditi are displayed, with amounts of 1500 and 3500.**

**Practical Proof:**




# Question 9: Explain the Difference Between Primary Key and Foreign Key

**Answer:**

**A Primary Key is a column or a combination of columns that uniquely identifies each record in a table. It does not allow duplicate or NULL values.**

**A Foreign Key is a column or a combination of columns that establishes a relationship between two tables. It refers to the primary key or another eligible unique key in another table and helps maintain referential integrity.**

**Difference Between Primary Key and Foreign Key:**

| Primary Key | Foreign Key |
| --- | --- |
| Uniquely identifies each record in a table. | Establishes a relationship between tables. |
| Does not allow duplicate values. | Can contain duplicate values. |
| Does not allow NULL values. | Can allow NULL values unless restricted. |
| Each table can have only one primary key constraint. | A table can have multiple foreign key constraints. |
| Ensures each record can be uniquely identified. | Ensures referenced records maintain referential integrity. |

**Example:**

**Consider two tables: `departments` and `employees`.**

**Departments Table**

| department_id (Primary Key) | department_name |
| --- | --- |
| 1 | HR |
| 2 | Sales |
| 3 | IT |

**Employees Table**

| employee_id (Primary Key) | employee_name | department_id (Foreign Key) |
|---|---|---|
| 101 | Amit | 1 |
| 102 | Riya | 2 |
| 103 | Raj | 3 |

In this example, `department_id` in the `departments` table is the primary key. In the `employees` table, `department_id` is a foreign key that refers to the department associated with each employee.

Conclusion:

A primary key uniquely identifies records within a table, while a foreign key connects related tables and helps maintain data consistency.

# Question 10: Explain Different Types of SQL Constraints

Answer:

SQL constraints are rules applied to table columns to maintain the accuracy, reliability, and integrity of data in a database.

The main types of SQL constraints are:

1. PRIMARY KEY

A primary key uniquely identifies each record in a table. It does not allow duplicate or NULL values.

Example:

employee_id INT PRIMARY KEY

2. FOREIGN KEY

A foreign key establishes a relationship between two tables by referencing a key in another table. It helps maintain referential integrity.

Example:

**FOREIGN KEY (department_id)**

**REFERENCES departments(department_id)**

**3. NOT NULL**

**The NOT NULL constraint ensures that a column cannot contain NULL values.**

**Example:**

**first_name VARCHAR(50) NOT NULL**

**4. UNIQUE**

**The UNIQUE constraint ensures that all values in a column, or combination of columns, are unique.**

**Example:**

**email VARCHAR(100) UNIQUE**

**5. CHECK**

**The CHECK constraint ensures that values satisfy a specified condition.**

**Example:**

**salary INT CHECK (salary > 0)**

**6. DEFAULT**

**The DEFAULT constraint assigns a predefined value to a column when no value is provided for it during insertion.**

**Example:**

**department VARCHAR(50) DEFAULT 'HR'**

**Conclusion:**

**SQL constraints help maintain data accuracy, consistency, and integrity. They prevent invalid data from being stored and ensure that database rules are followed.**